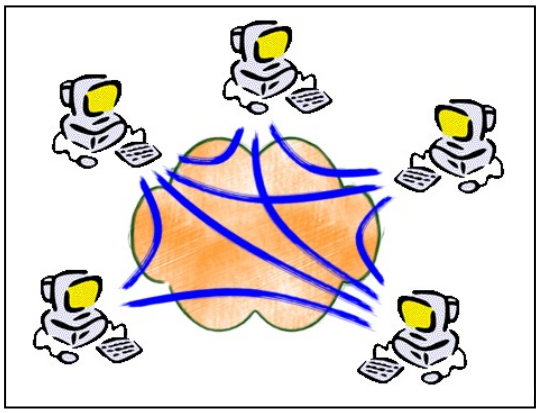
Prof. Dr. Erwin Mayer

Fakultät für Elektrotechnik
Medizin und Informatik

Praktikum Verteilte Systeme

Versuch 1 (2 Labortermine)

Peer-to-Peer Computing



Was Sie in diesem Versuch lernen:

- Vertrautwerden mit einer einfachen verteilten Anwendung
- Realisierung eines einfachen Peer-to-Peer Multi-Player Computer Games
- Aufbau eines Peer-to-Peer Gruppenmanagements
- Diskussion von Entwurfsentscheidungen für verteilte Systeme
- Absprache von Schnittstellen im Team und verteilte Komponentenentwicklung
- Testen von verteilten Anwendungen

Voraussetzung zur Teilnahme: Pre-Lab Aufgaben komplett durchgeführt (mit Nachweis) !

1 PRE-LAB-Fragen

Name	Matrikelnummer	Semester
Nadine Bichler	192487	A16

1. Was ist der prinzipielle Unterschied zwischen verteilten Anwendungen, die nach dem Peer-to-Peer-Prinzip arbeiten und solchen, die das Client-Server-Prinzip anwenden.

Client-Server:
- server = Dienstanbieter
- client = Dienstnehmer
- klare Rollenverteilung

P2P:
- alle sind Client & Server zugleich
- können mit allen kommunizieren

2. Angenommen ein Prozess muss gleichzeitig auf eine Benutzereingabe von der Konsole und auf das Eintreffen einer Nachricht warten. Wie realisieren Sie dies?

Multithreading

3. Wie realisieren Sie das Empfangen einer einzelnen UDP-Nachricht über eine Socket-Schnittstelle mit der Programmiersprache Ihrer Wahl (Java, Python, C, C++, C#). Bitte auch alle vorbereitenden Schritte.

```c
//client
int main()
{
    //address
    struct sockaddr_in serveraddr;
    serveraddr.sin_family = AF_INET;
    serveraddr.sin_port = htons(9999);
    serveraddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    unsigned int serverlen = sizeof(struct sockaddr);

    //socket
    int socket_id = socket(AF_INET, SOCK_DGRAM, 0);
    if (socket_id == -1)
    {
        perror("error socket\n");
    }
    else
    {
        printf("socket success\n");
    }

    char myMessage[6] = "hello";
    size_t messageLength = 6

    //sendto
    int sendto_id = sendto(socket_id, myMessage, messageLength, 0, (struct sockaddr*)&serveraddr, serverlen);
    if (sendto_id == -1)
    {
        perror("error sendto\n");
    }
    else
    {
        printf("sendto success\n");
    }

    //close
    int close_id = close(socket_id);
    if (close_id == -1)
    {
        perror("error close\n");
    }
    else
    {
        printf("close success\n");
    }
}
```

```c
//server
int main()
{
    //my address
    struct sockaddr_in myaddr;
    myaddr.sin_family = AF_INET;
    myaddr.sin_port = htons(9999);
    myaddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    //client
    unsigned int clientLen = sizeof(struct sockaddr);
    struct sockaddr_in clientAddr;

    char buffer[100];


    //socket
    int socket_id = socket(AF_INET, SOCK_DGRAM, 0);
    if (socket_id == -1)
    {
        perror("error socket\n");
    }
    else
    {
        printf("socket succes\n");
    }

    //bind
    int bind_id = bind(socket_id, (struct sockaddr*)&myaddr, sizeof(myaddr));
    if (bind_id == -1)
    {
        perror("error bind\n");
    }
    else
    {
        printf("bind succes\n");
    }

    //recvfrom
    int recvfrom_id = recvfrom(socket_id, buffer, sizeof(buffer) - 1, 0, (struct sockaddr*)&clientAddr, &clientLen);
    if (recvfrom_id == -1)
    {
        perror("error recvfrom\n");
    }
    else
    {
        printf("recvfrom success\n");
        printf(buffer);
    }
}
```

4. Wie erzeugen Sie eine Nachricht mit einem Integer und einem String als Inhalt mit einer Programmiersprache Ihrer Wahl (Java, Python, C/C#/C++). Verwenden Sie, wo möglich, JSON-basierte Nachrichten.

```
int main(){
  int id=42;
  int *str = "hello";
  printf("{\"id\":%d, \"message\": \"%s\"}\n", id, str);
}
```

5. Wie erzeugen Sie eine Nachricht mit einem Integer und einer Liste mit Strings als Inhalt mit einer Programmiersprache Ihrer Wahl (Java, Python, C/C#/C++). Verwenden Sie, wo möglich, JSON-basierte Nachrichten.

```
int main() {
  printf("{\"id\":42, \"messages\": [\"Hallo\", \"Welt\", \"Test\"]}\n");
}
```

6. Was könnte eine Datenstruktur aussehen, welche bei einem "4-Gewinnt" Multi-Player-Spiel den auf allen Peers replizierten SPIELZUSTAND zu einem gegebenen Zeitpunkt wiedergibt?

```
struct {
 "board": [
  [0,0,0,0,0,0,0],
  [0,0,0,0,0,0,0],
  [0,0,0,1,0,0,0],
  [0,0,2,1,0,0,0],
  [0,2,1,2,0,0,0],
  [1,1,2,1,2,0,0]
 ],
 "currentPlayer": 2,
 "players": [1,2],
 "lastMove": { "column": 3, "row": 2, "player": 1 },
 "moveCount": 10
};
```